

University of Asia Pacific
Department of Computer Science and Engineering

CSE 402: Operating System Lab

Lab Report: 06

Disk Scheduling Analysis: FCFS vs. SSTF with Initial Head Bypass

Submitted By:

Name: Faiaz Masnun Labib
Student ID: 23101128
Section: C-1

Submitted To:

Atia Rahman Orthi
Lecturer,
Department of CSE, UAP

Date of Submission: October 12, 2026

1. Problem Specification

- **Objective:** Calculate and compare Total Head Movement (Total Cost) for FCFS and SSTF disk scheduling algorithms.
- **Workload Parameters:**
  - Request Queue (req): $[0, 30, 41, 62, 14, 20]$
  - Initial Head Position (h): 20
- **Implementation Rule:** If a request in the queue matches the initial head position ($t == h$), it is bypassed in FCFS as already serviced at the initial location.
- **Cost Metric Formula:**

$$\text{Total Movement (Cost)} = \sum |t_{\text{target}} - c_{\text{current}}|$$

Request Index	0	1	2	3	4	5
Track Number	0	30	41	62	14	20

Table 1: Disk Track Request Input

2. Source Code

```
def fcfs(req, h):
    tmv = 0
    c = h

    for t in req:
        if t == h:
            continue
        tmv += abs(t - c)
        c = t

    return tmv


def sstf(req, h):
    tmv = 0
    c = h
    rem = req.copy()
    seq = []

    while rem:
        clos = min(rem, key=lambda t: abs(t - c))
        tmv += abs(clos - c)
        c = clos
        seq.append(clos)
        rem.remove(clos)

    return tmv


```

```
req = [0, 30, 41, 62, 14, 20]
h = 20

print(f"FCFS total cost: {fcfs(req, h)}")
print(f"SSTF total cost: {sstf(req, h)}")
```

GitHub Repository Link: Click Here

3. Step-by-Step Calculation Trace & Results

A. FCFS Step-by-Step Trace

Since track 20 matches the initial head ($t == h$), it is bypassed by the guard check:

- Track 0: $|0 - 20| = 20$, head moves to $c = 0$
- Track 30: $|30 - 0| = 30$, head moves to $c = 30$
- Track 41: $|41 - 30| = 11$, head moves to $c = 41$
- Track 62: $|62 - 41| = 21$, head moves to $c = 62$
- Track 14: $|14 - 62| = 48$, head moves to $c = 14$
- Track 20: $t == h$ ($20 == 20$), bypassed by `continue`.
- **Execution Sequence:** $20 \to 0 \to 30 \to 41 \to 62 \to 14$

Step	**From Track (c)**	**To Track (t)**	**Head Movement**
1	20	0	$\|0 - 20\| = 20$
2	0	30	$\|30 - 0\| = 30$
3	30	41	$\|41 - 30\| = 11$
4	41	62	$\|62 - 41\| = 21$
5	62	14	$\|14 - 62\| = 48$
6	14	20	Bypassed ($t == h$)
FCFS Total Cost			**130**

B. SSTF Step-by-Step Trace

Selects the closest available track dynamically:

- At $c = 20$: Minimum distance is track 20 ($|20 - 20| = 0$), $c = 20$.
- At $c = 20$: Remaining $\{0, 14, 30, 41, 62\}$. Closest is 14 ($|14 - 20| = 6$), $c = 14$.
- At $c = 14$: Remaining $\{0, 30, 41, 62\}$. Closest is 0 ($|0 - 14| = 14$), $c = 0$.
- At $c = 0$: Remaining $\{30, 41, 62\}$. Closest is 30 ($|30 - 0| = 30$), $c = 30$.
- At $c = 30$: Remaining $\{41, 62\}$. Closest is 41 ($|41 - 30| = 11$), $c = 41$.
- At $c = 41$: Remaining $\{62\}$. Closest is 62 ($|62 - 41| = 21$), $c = 62$.
- **Execution Sequence:** $20 \to 20 \to 14 \to 0 \to 30 \to 41 \to 62$

C. Output Verification

Step	From Track (c)	To Track (clos)	Head Movement
1	20	20	$\|20 - 20\| = 0$
2	20	14	$\|14 - 20\| = 6$
3	14	0	$\|0 - 14\| = 14$
4	0	30	$\|30 - 0\| = 30$
5	30	41	$\|41 - 30\| = 11$
6	41	62	$\|62 - 41\| = 21$
SSTF Total Cost			**82**

Algorithm	Total Cost (Head Movement)	Status
FCFS	130	High Seek Overhead
SSTF	82	**Better (Optimal)**

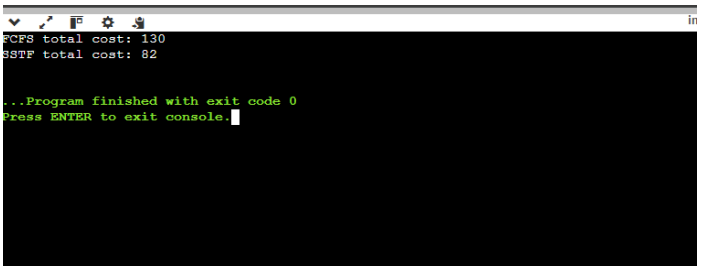


Figure 1: Program Execution Output Verification

4. Comparative Evaluation: Which Approach Performs Better?

Decision: SSTF (Shortest Seek Time First) is significantly better than FCFS for this dataset.

- **Performance Gain:**

$$\Delta\text{Cost} = 130 - 82 = \mathbf{48} \text{ cylinders reduction}$$

$$\text{Percentage Improvement} = \frac{130 - 82}{130} \times 100\% = \mathbf{36.92}\%$$

- **Behavioral Comparison:**
  - **FCFS Path Inefficiencies:** FCFS suffers from wild oscillations across the disk surface ($0 \to 30 \to 41 \to 62 \to 14$). The backward jump from track 62 to track 14 wastes 48 cylinders of seek time alone.
  - **SSTF Directional Optimization:** SSTF greedily satisfies localized requests first ($20 \to 14 \to 0$), and then sweeps upward monotonically ($0 \to 30 \to 41 \to 62$), eliminating unnecessary actuator reversals.

5. Discussion

- **Handling the Initial Head Position:**

  - In FCFS, the condition `if t == h: continue` skips the request at index 5 because the head was already stationed at track 20 initially. Without this skip, returning from track 14 back to 20 would have added an extra 6 cylinders ($130 + 6 = 136$).
  - In SSTF, request 20 is picked first with a cost of 0 ($|20 - 20| = 0$), naturally clearing it without needing a manual bypass check.

- **Mechanical Overhead:** In magnetic hard disk drives, seek time accounts for the largest component of I/O latency. Reducing the total seek distance from 130 to 82 directly translates to faster disk throughput and decreased physical actuator wear.

- **Conclusion:** For this request queue, SSTF achieves a 36.92% performance improvement over FCFS. While SSTF carries a theoretical risk of starvation for requests on outer tracks under continuous heavy workloads, for batch processing tasks it provides superior seek minimization.